

BRAND ENGINE

Backend Architecture & Technical Overview

Current implementation baseline - August 2026

1. Project Overview

Brand Engine is a modular Java backend application for managing organizations, projects, users, documents, uploaded files, and AI-assisted workflows. It is implemented as a REST API with Spring Boot and PostgreSQL, with emphasis on separation of concerns, secure stateless authentication, database migration discipline, automated testing, and extensibility for AI features.

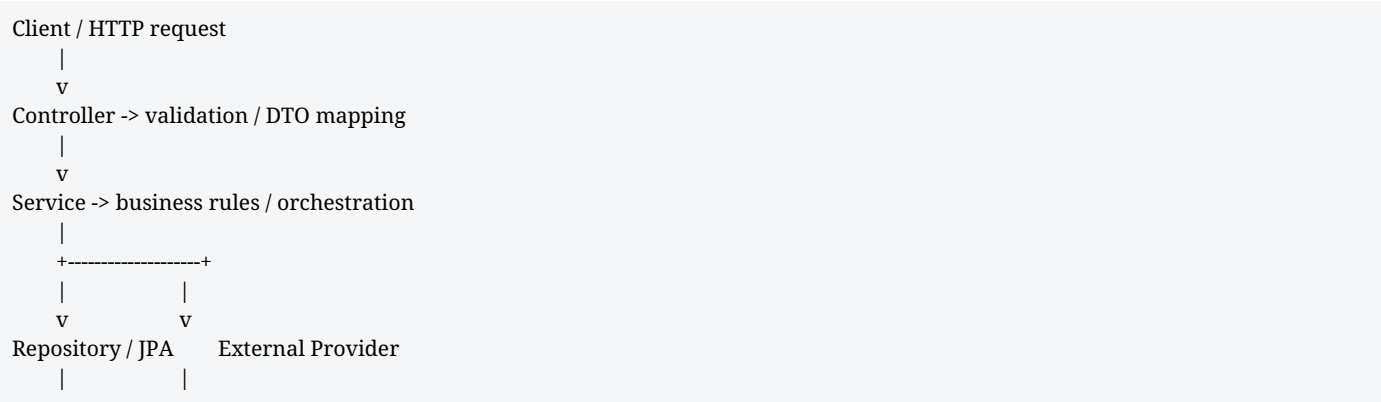
Primary purpose: provide an authenticated backend where users can organize project data, upload documents, and invoke AI services through a provider-agnostic integration layer.

2. Technology Stack

Area	Implementation
Runtime	Java 21
Framework	Spring Boot 4.1.0
Web	Spring Web MVC
Persistence	Spring Data JPA / Hibernate
Database	PostgreSQL
Migrations	Flyway
Security	Spring Security + JWT; stateless sessions
HTTP client	Spring WebClient
AI	OpenAI Responses API through AIProvider
Build	Maven
Testing	JUnit 5, Mockito, MockMvc

3. High-Level Architecture

Brand Engine follows a layered modular architecture. Controllers own HTTP concerns, services own business rules and orchestration, repositories isolate persistence, and DTOs define API contracts. Security, configuration, and exception handling are cross-cutting concerns.



4. Main Modules

- **organization:** Organization CRUD and persistence.
- **project:** Project CRUD, organization association, validation, and lookup.
- **document:** Document metadata CRUD, project-scoped access, multipart upload, and file-storage metadata.
- **user:** Authenticated profile retrieval, email update, and password change.
- **auth:** Registration, login, JWT generation/validation, authentication filter, and stateless security.
- **ai:** AI chat endpoint, provider abstraction, OpenAI implementation, configuration, and AI-specific error handling.
- **common/config/exception:** Shared API responses, application configuration, and centralized exception mapping.

5. Security Architecture

Spring Security is configured for stateless authentication. Registration and login are public; protected application endpoints require a valid JWT. A custom JWT authentication filter executes before the username/password authentication filter.

- SessionCreationPolicy.STATELESS is used; server-side login sessions are not required.
- CSRF is disabled for the stateless bearer-token REST API.
- Clients send Authorization: Bearer <token> on protected endpoints.
- The OpenAI API key is supplied through the OPENAI_API_KEY environment variable and is not stored in source code.

6. Data & Persistence

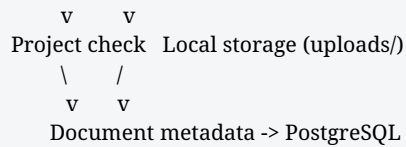
PostgreSQL is the system of record. Spring Data JPA repositories provide persistence access, Flyway manages schema evolution, and Hibernate validates the schema rather than silently creating application tables.

```
Organization
|
+-- Projects
|
+-- Documents
|
+-- uploaded-file metadata
    (filename, content type, storage key, size, timestamps)
```

7. Document Upload Architecture

The document module supports multipart uploads. The controller accepts projectId, name, and MultipartFile. The service verifies the parent project, stores the file under a generated storage key, and persists document metadata.

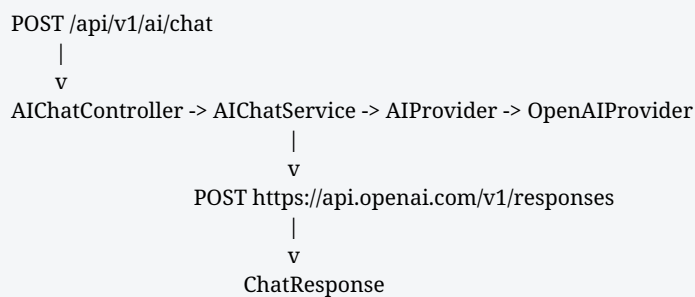
```
multipart/form-data (projectId, name, file)
|
▼
DocumentController
|
▼
DocumentService
/  \
```



8. AI Integration

The AI module applies dependency inversion so business code is not coupled directly to OpenAI. AIChatService depends on the AIProvider interface; OpenAIProvider is one implementation and uses WebClient to call the OpenAI Responses API.

- AIProvider defines the chat contract.
- MockAIProvider supports local/testing behavior without a real external call.
- OpenAIProvider constructs requests, parses output text, and wraps upstream HTTP failures.
- AIServiceException represents external AI failures.
- GlobalExceptionHandler maps AI service failures to a safe 502 Bad Gateway response.



9. Error Handling

GlobalExceptionHandler converts known failures into predictable API responses: validation -> 400, missing domain records -> 404, conflicts -> 409, upstream AI failures -> 502, and unexpected failures -> logged 500 responses.

10. Testing Strategy

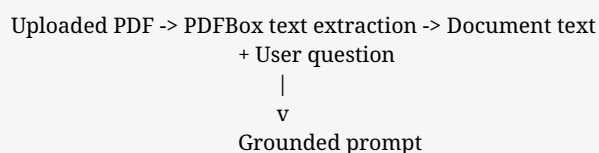
The project currently maintains 79 passing automated tests across multiple layers.

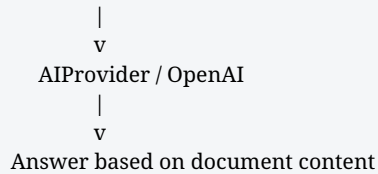
- Controller tests with MockMvc for HTTP contracts and validation.
- Service tests with Mockito for business logic and dependency interactions.
- Repository tests for persistence behavior.
- OpenAI provider tests for success parsing, empty output, and upstream API errors.
- Exception-handler test for AI failure mapping.

11. Current AI-RAG Development Direction

Status: The base AI chat integration is complete. Retrieval-augmented generation (RAG) is the next feature and is not yet part of the completed baseline.

The first RAG slice will extract text from an uploaded PDF and use that content as context when answering a user question.





A later RAG iteration can add chunking, embeddings, and vector retrieval for efficient search across larger documents and collections.

12. Architectural Principles

Principle	How Brand Engine Applies It
Separation of concerns	Controllers, services, repositories, DTOs, configuration, and integrations have distinct responsibilities.
Dependency inversion	AI business logic depends on AIProvider rather than a specific vendor implementation.
Secure configuration	Secrets are injected through environment variables rather than committed to source control.
Migration-first persistence	Database schema changes are managed with Flyway.
Testability	Constructor injection makes dependencies easy to mock and verify.
Incremental delivery	Features are developed in dedicated Git branches and merged after verification.

End of architecture overview